Utah State University Hackathon

Databricks Data App Factory

Administrative Instructional Document & Workshop Framework

24 hour hackathon Event Details:

Dates: February 27-28, 2026 (24 Hours)
Location: Utah State University, Logan, Utah
Participants: 100 Students
Team Structure: 3-4 students per team (25-33 teams)

📋 Table of Contents

1. Executive Summary
2. Hackathon Challenge Overview
3. Pre-Hackathon Preparation
4. Databricks Free Edition Setup
5. Foundational Learning Resources
6. Workshop Framework & Schedule
7. Technical Requirements & Guidelines
8. Success Criteria & Evaluation
9. Presentation & Demo
10. Support Resources

1. Executive Summary

Welcome to the HackUSU Databricks Data App Factory! This workshop challenges students to revolutionize how business users interact with data by building governed, self-service data applications.

The Vision

Empower non-technical business users to deploy data applications without needing data engineering or software development expertise, while maintaining enterprise-grade security, governance, and architectural standards.

The Approach

Combine an AI IDE (Kiro or similar) with Databricks Apps to translate natural language intent into secure, governed data applications that provide full IT visibility and control while enabling business self-service.

---

2. Hackathon Challenge Overview

🎯 Core Challenge

"How do we empower the average business user to deploy governed data applications without needing to be a data engineer or software developer?"

🔑 Key Components

The Data App Factory Concept

Students will develop a framework that includes:

1. Steering Documents: Define application templates, patterns, and guardrails
2. Governance Framework: Establish security policies, data access controls, and compliance rules
3. AI Translation Layer: Use natural language processing to convert business requirements into technical specifications
4. Automated Deployment: Create sandbox environments with full IT oversight

Success Criteria: Your Data App Must

✅ Pull from Governed Data Sources

- Connect to Unity Catalog tables with proper permissions
- Demonstrate row-level security or column masking
- Show audit trail capabilities

✅ Visualize Key Metrics & KPIs

- Display interactive dashboards
- Include at least 3-5 meaningful visualizations
- Provide filtering and drill-down capabilities

✅ Include Conversational Interface

- Embed a chatbot that allows users to "chat with data"
- Support natural language queries (e.g., "What were sales last quarter?")
- Return data-driven answers with visualizations

🎁 Value Proposition

Make building data apps as easy for business users as creating dashboards, while accelerating value creation and maintaining security, governance, and architectural standards.

---

3. Pre-Hackathon Preparation

📅 Timeline: 1-2 Weeks Before Event

All participants must complete the following preparation before the hackathon begins:

✅ Mandatory Pre-Work Checklist

- Register for Databricks Free Edition account
- Complete "Databricks Fundamentals" (2 hours)
- Complete "Generative AI Fundamentals" (1 hour)
- Watch: "Getting Started with Databricks Apps" tutorial
- Review: Databricks Apps documentation
- Form your team (3-4 members)
- Identify team roles (see below)

👥 Recommended Team Composition

Each team of 3-4 students should include:

Role	Responsibilities	Skills Needed
Data Engineer	Set up data sources, manage Unity Catalog, handle governance	SQL, data modeling
App Developer	Build Databricks App, create UI/UX, integrate components	Python, Streamlit/Gradio, web development

AI/ML Specialist	Implement conversational interface, integrate LLM capabilities	Python, LLM frameworks, APIs
Business Analyst (optional for 4-person teams)	Define use case, design KPIs, test user experience	Business domain knowledge, requirements gathering

Note: Roles can overlap depending on team size and skill sets.

---

4. Databricks Free Edition Setup

🚀 Registration Process

Step 1: Sign Up for Databricks Free Edition

1. Navigate to Databricks Free Edition
    - Go to:
    - https://www.databricks.com/try-databricks/free-edition
    - Or search "Databricks Free Edition" in your browser
2. Create Your Account
    - Click "Sign up for Free Edition"
    - Enter your email address (use your USU email: @usu.edu or @aggiemail.usu.edu)
    - Create a secure password
    - Verify your email address
3. Complete Registration
    - Fill in basic information (name, organization: "Utah State University")
    - Select your use case: "Learning/Education"
    - Accept terms and conditions
    - Click "Create Account"

4. Workspace Provisioning
    - Databricks will automatically provision your workspace (takes 2-3 minutes)
    - You'll be redirected to your personal workspace
    - Your workspace URL will be: https://[your-workspace-id].cloud.databricks.com

Step 2: Verify Your Workspace

Once logged in, you should see:

- Left sidebar navigation with: Workspace, Catalog, Compute, Workflows, Apps
- Top menu with: +New, Recent, Search
- Main workspace area

🎉 Congratulations! Your Databricks Free Edition is ready!

📊 Free Edition Resource Limitations

Understanding these limitations is critical for team planning:

Additional credits are available via request (provisioned typically within 1 hr)

Resource	Limitation	Impact on Teams
Compute	Limited to small cluster sizes	Sufficient for prototyping; avoid large datasets (>100MB)
SQL Warehouses	One 2X-Small warehouse per account	Teams should share queries efficiently

Concurrent Jobs	Max 5 concurrent job tasks	Limit background processing
Storage	Serverless only, quota-limited	Use sample datasets or small production data samples
Users	Single user per Free Edition account	Each team member needs their own account
Apps	Limited but sufficient for demo apps	Can deploy 1 app per account
Pipelines	One active pipeline per type	Focus on simple data flows

🤝 Team Collaboration Strategy

Since Free Edition is single-user, teams must coordinate differently:

Option 1: Designated App Builder (Recommended for Hackathon)

- ONE team member hosts the final application
- Other team members develop components separately
- Use Git/GitHub for code sharing
- Deploy final app from one account for demo

Option 2: Distributed Development

- Each team member builds components in their own workspace

- Share notebooks via GitHub or export/import
- Integrate components into one workspace before presentation

Option 3: Trial Account for Teams (If Available)

- One team member can sign up for a 14-day Databricks trial (full version)
- Provides multi-user workspace capabilities
- Requires credit card (won't be charged if cancelled before 14 days)
- Use for final integration only

⚠️ Critical Setup Notes

1. Create Accounts Early: Don't wait until hackathon day
2. Test Access: Verify you can log in and create a notebook
3. Bookmark Workspace URL: Save your workspace link for quick access
4. Share Workspace IDs: Team members should know each other's workspace URLs for collaboration
5. Backup Your Work: Export notebooks regularly to prevent data loss

---

5. Foundational Learning Resources

🎓 Pre-Requisite Training (Mandatory)

Complete these before the hackathon begins:

1. Databricks Fundamentals Accreditation (FREE)

Duration: ~2 hours
Link:

https://www.databricks.com/learn/training/home

What You'll Learn:

- Databricks workspace navigation
- Notebooks and collaborative development
- Unity Catalog basics

- Data management fundamentals

How to Access:

1. Go to Databricks Training homepage
2. Search for "Databricks Fundamentals"
3. Enroll for free
4. Complete modules and assessment
5. Earn your badge (share on LinkedIn!)

---

2. Generative AI Fundamentals Accreditation (FREE)

Duration: ~1 hour
Link:

https://www.databricks.com/learn/training/home

What You'll Learn:

- Generative AI concepts and use cases
- Large Language Models (LLMs)
- Prompt engineering basics
- AI governance and ethics

How to Access:

1. Go to Databricks Training homepage
2. Search for "Generative AI Fundamentals"
3. Complete the self-paced course
4. Pass the assessment
5. Receive certification badge

---

3. AI Agent Fundamentals Accreditation (FREE)

Duration: ~1.5 hours
Link:

https://www.databricks.com/learn/training/home

What You'll Learn:

- AI agent architecture and workflows
- Building conversational interfaces
- Integrating agents with data sources
- Best practices for agent development

📚 Recommended Databricks Academy Courses

These courses provide deeper knowledge (optional but highly recommended):

Course	Duration	Focus Area	Link
Data Engineering with Databricks	4-6 hours	ETL pipelines, Delta Lake, data governance	Databricks Academy
Machine Learning with Databricks	3-4 hours	ML workflows, MLflow, model serving	Databricks Academy
Lakehouse Fundamentals	2 hours	Data architecture, Unity Catalog	Databricks Academy
SQL for Data Analysis	2-3 hours	SQL queries, visualizations, dashboards	Databricks Academy

Access Instructions:

1. Log in to your Databricks workspace
2. Click your profile icon → "Learning"
3. Browse Databricks Academy courses
4. Filter by "Free" courses
5. Enroll and start learning

🎬 Essential Video Tutorials

Watch these before and during the hackathon:

Databricks Apps Tutorials

1. "Getting Started with Databricks Apps" (Official Documentation)
    - Link:
    - https://docs.databricks.com/aws/en/dev-tools/databricks-apps/get-started
    - Duration: 15-minute read
    - Content: Step-by-step app creation, deployment, local development
    - Short highlevel video overview building databrics apps: https://www.databricks.com/resources/demos/videos/databricks-apps-introduction
2. "Deploy AI Chatbots in Minutes with Databricks Apps"
    - Platform: YouTube
    - Link:
    - https://www.youtube.com/watch?v=Wg9w4MBpXw0
    - Duration: ~10 minutes
    - Content: Building conversational interfaces with pre-built templates
3. "Build Quick Apps in Databricks - Streamlit, Gradio, Dash, Flask"
    - Platform: YouTube
    - Link:
    - https://www.youtube.com/watch?v=mgdu_YhlVr8
    - Duration: ~15 minutes
    - Content: Hands-on demo of creating apps with different frameworks
4. "Create a RAG based Chatbot with Databricks"
    - Platform: YouTube
    - Link:
    - https://www.youtube.com/watch?v=p4qplgj5Zjg
    - Duration: ~20 minutes
    - Content: Implementing Retrieval Augmented Generation for data chatbots

5. "Learn Databricks for Free in 2026 | End-To-End Learning Path"
    - Platform: YouTube
    - Link:
    - https://www.youtube.com/watch?v=gV2zCoE1Xss
    - Duration: ~10 minutes
    - Content: Comprehensive overview of free resources

📖 Key Documentation & White Papers

Must-Read Documentation

1. Databricks Apps Overview
    - https://docs.databricks.com/aws/en/dev-tools/databricks-apps/
    - Conceptual overview, requirements, limitations
2. Databricks Apps Development Guide
    - https://docs.databricks.com/aws/en/dev-tools/databricks-apps/get-started
    - Complete development workflow
3. AI/BI Genie Space Documentation
    - https://docs.databricks.com/aws/en/genie/
    - Natural language interface for data
4. Genie Conversation API
    - https://docs.databricks.com/aws/en/genie/conversation-api
    - Integrate conversational AI into your apps
5. Unity Catalog Overview
    - https://docs.databricks.com/en/data-governance/unity-catalog/
    - Data governance and security

Recommended Blog Posts & Case Studies

1. "Introducing Databricks Apps"
    - https://www.databricks.com/blog/introducing-databricks-apps

    - Official launch announcement with use cases
2. "Building a Chatbot with Large Language Models (LLMs)"
    - https://www.databricks.com/solutions/accelerators/building-a-chatbot-with-large-language-models
    - Solution accelerator with code samples
3. "Implementing a RAG Chatbot using Databricks and Pinecone"
    - https://www.databricks.com/blog/implementing-rag-chatbot-using-databricks-and-pinecone
    - Real-world implementation guide
4. "Mastering Databricks Apps: From Development to Secure Deployment"
    - https://www.tredence.com/blog/mastering-databricks-apps-from-creation-to-deployment-of-interactive-data-applications
    - Third-party comprehensive guide

🛠️ Hands-On Practice Exercises

Before the hackathon, try these exercises:

Exercise 1: Hello World App (30 minutes)

1. Create your first Databricks App using Gradio template
2. Deploy and access via URL
3. Modify the app locally and redeploy

Exercise 2: Data Connection (45 minutes)

1. Create a sample table in your workspace
2. Query the table from a notebook
3. Display results in a simple Streamlit app

Exercise 3: Simple Dashboard (1 hour)

1. Load sample sales data
2. Create 3 visualizations (bar chart, line chart, metrics)
3. Deploy as a Databricks App

Exercise 4: Basic Chatbot (1 hour)

1. Use Databricks chatbot template
2. Deploy the template
3. Test conversational interface
4. Understand the code structure

📝 Pre-Hackathon Assessment

Test your readiness with this self-assessment:

Knowledge Check:

- I can create and run a Databricks notebook
- I understand Unity Catalog and data governance
- I know what Databricks Apps are
- I can explain what a conversational interface is
- I understand the concept of steering documents
- I'm familiar with at least one Python web framework (Streamlit/Gradio/Dash/Flask)

Practical Skills:

- I've created at least one Databricks App
- I've queried data using SQL in Databricks
- I've created a simple visualization
- I've worked with Python in Databricks notebooks

Team Readiness:

- My team is formed (3-4 members)
- We've assigned roles
- We've discussed our approach
- We've identified a potential use case

If you checked less than 70% of these boxes, allocate more pre-hackathon prep time!

6. Workshop Framework (Presentation & Demo)

📅 Schedule - Friday, February 27, 2026

🌅 Databricks Apps Session

"The Data App Factory Vision" (*5 min*)

- Why business user empowerment matters
- The role of AI in democratizing app development
- Real-world case study example

Databricks Apps Deep Dive

Part 1: Architecture & Components (10 min)

- Databricks Apps architecture overview
- Framework comparison (Streamlit vs Gradio vs Dash vs Flask)
- Unity Catalog : Security and governance

Part 2: Live Demo - Building Your First Data App (30 min)

- Step-by-step app creation
- Connecting to data sources
- Creating visualizations
- Deploying and sharing

Conversational Interfaces:

- Genie API overview
- Implementing chatbots in apps
- Best practices for natural language queries

🌆 Extended Content Sessions Concepts (TBD)

Workshop - Governance & Steering Documents

- *Creating steering documents for app templates*
- *Defining guardrails and policies*
- *Implementing automated governance*
- *Sample templates walkthrough*

Kiro AI IDE Introduction (Optional)

- *Overview of AI-powered IDEs*
- *Natural language to code translation*
- *Integration with Databricks*
- *Demo and Q&A*

Hacking Begins! 🚀

---

7. Technical Requirements & Guidelines

🛠️ Technical Stack

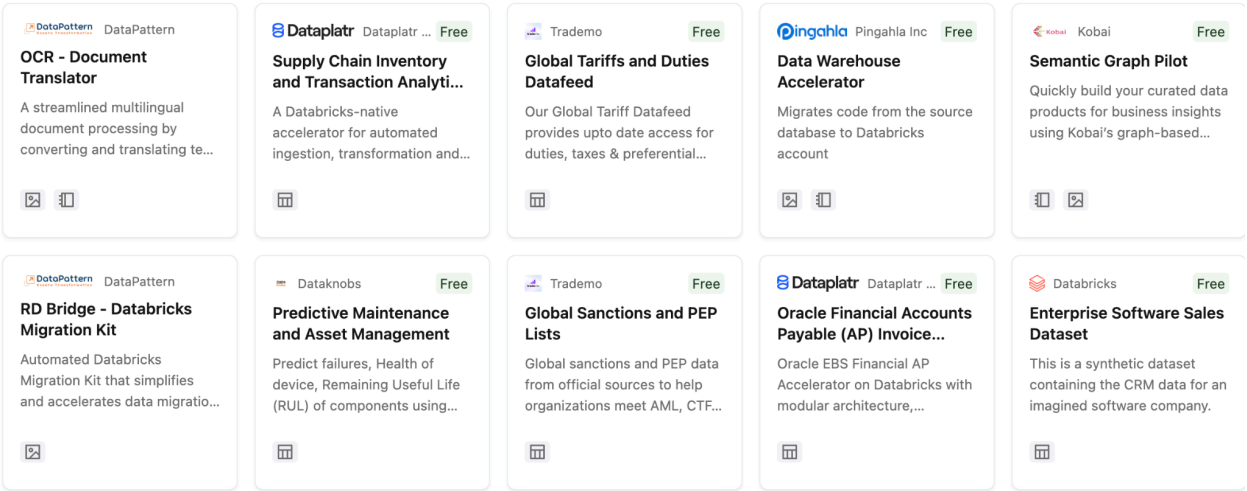
Required Components

1. Databricks Platform
    - Databricks Free Edition workspace
    - Unity Catalog (for governance)
    - SQL Warehouse (for queries)
    - Databricks Apps (for deployment)
2. Application Framework (Choose ONE)
    - Streamlit (Recommended for beginners) - Python-based, simple syntax
    - Gradio (Recommended for ML apps) - Great for chatbots
    - Dash - More complex, powerful dashboards
    - Flask - Full control, requires more coding
    - Shiny - For R users

3. Data Layer
    - At least one governed table in Unity Catalog
    - Sample datasets acceptable (provided by organizers)
    - Option to use public datasets
4. Conversational Interface
    - Genie API for natural language queries
    - OR custom chatbot using LLM APIs
    - OR rule-based chatbot for simple interactions

📊 Sample Datasets Provided: [Databricks Marketplace]

Teams can choose from these free datasets available in Databricks Marketplace (re: Manufacturing) or bring their own::



- Datasets will be shared in a common Unity Catalog
- Teams can create secure views with their own access controls
- Synthetic data (no real PII)

🎯 Application Requirements

Your Data App Factory solution should include:

Core Features

1. Governed Data Access
    - Connect to Unity Catalog tables
    - Demonstrate data lineage
    - Show at least one security feature (row-level security, column masking, or access audit)
2. Visual Dashboard
    - Minimum 3 different visualization types (e.g., bar chart, line chart, pie chart)
    - At least 5 KPIs or metrics
    - Interactive elements (filters, drill-downs, date ranges)
3. Conversational Interface
    - Text-based chat interface
    - Ability to ask questions in natural language
    - Return data-driven answers
    - Display results as text AND visualizations
4. Steering Documents
    - Document your app template structure
    - Define guardrails (data access, security policies)
    - Create user guide for business users
5. Governance Framework
    - Define roles and permissions
    - Establish data quality rules
    - Create approval workflow (can be conceptual)

Advanced Features

- Multi-page application with navigation
- Real-time data updates
- Export functionality (CSV, PDF)
- User authentication and personalization
- Mobile-responsive design
- AI-generated insights or recommendations
- Natural language app generation (user describes app → system generates it)
- Integration with external APIs
- Predictive analytics or ML models
- Workflow orchestration

🔒 Governance & Security Guidelines

Your solution should demonstrate these governance principles:

1. Data Governance

- Data Discovery: Users can search for approved datasets
- Data Lineage: Track data sources and transformations
- Data Quality: Validate data before visualization
- Data Classification: Label sensitive data appropriately

2. Access Control

- Role-Based Access: Define user roles (viewer, analyst, admin)
- Row-Level Security: Different users see different data subsets
- Column Masking: Hide or mask sensitive columns (PII)
- Audit Logging: Track who accessed what data when

3. Application Guardrails

- Template Restrictions: Limit app types business users can create
- Resource Limits: Prevent over-consumption of compute
- Approval Workflow: Apps require IT review before production
- Sandbox Environment: Users deploy to sandboxes first

4. IT Oversight

- Visibility Dashboard: IT sees all deployed apps
- Health Monitoring: Track app performance and errors
- Compliance Reporting: Generate governance reports
- Kill Switch: Ability to disable non-compliant apps

💻 Development Best Practices

Code Organization

```text
my-data-app/
├── app.yaml                  # App configuration
├── requirements.txt          # Python dependencies
├── app.py                    # Main application file
├── data/
│   ├── queries.sql          # SQL queries
│   └── sample_data.csv      # Sample data for local testing
├── components/
│   ├── dashboard.py         # Dashboard components
│   ├── chatbot.py           # Chat interface
│   └── auth.py              # Authentication logic
├── governance/
│   ├── steering_doc.md      # App template documentation
│   ├── guardrails.yaml      # Security policies
│   └── roles.yaml           # User roles and permissions
├── utils/
│   ├── databricks_connect.py  # DB connection utilities
│   └── data_processing.py     # Data transformation functions
└── README.md                # Project documentation
```

Version Control

- Use Git/GitHub for source control
- Commit frequently with meaningful messages
- Tag your submission version

Testing

- Test with different user roles
- Validate with edge cases
- Test on different screen sizes

Documentation

- Clear README with setup instructions
- Inline code comments
- User guide for business users

🚫 Common Pitfalls to Avoid

1. Overcomplicating the Solution
    - Start simple, add complexity incrementally
    - Focus on core requirements first
2. Ignoring Governance
    - Governance is not optional - it's core to the challenge
    - Don't treat security as an afterthought
3. Poor User Experience
    - Business users are non-technical - make it intuitive
    - Test with someone unfamiliar with your app
4. Weak Conversational Interface
    - Chat should actually work, not just be a static form
    - Test with real questions business users would ask
5. Insufficient Steering Documents
    - Don't just describe your app - describe the template system
    - Show how business users would use YOUR framework to create THEIR apps
6. Data Connection Issues
    - Test data connections early
    - Have fallback sample data
7. Last-Minute Integration
    - Integrate components continuously

- Don't wait until the last hour

---

8. Success Criteria & Evaluation

🏆 Judging Criteria

Provided by [HackUSU]

Judging:

- During the judging phase, you will have 10 minutes to present your project and seek feedback from judges.
- Judges will determine beginner and advanced winners in each category.

Judging Criteria:

Technology:

- *How technically impressive was the project?*
- *Was the problem difficult?*
- *Did your team use a clever technique?*

Design:

- *How well designed is the user interface/experience?*
- *Are the graphics impressive?*
- *Completion: Does the project work? Did your team achieve what you wanted?*

***Learning**:*

- *Did team members try to learn something new?*
- *Did you attempt something different than you had worked on before? This is especially important for the beginner prize.*

9. Presentation & Demo

🎬 Demo Best Practices

Preparation

1. Test Everything
    - Demo in same environment you'll present from
    - Check internet connectivity
    - Have backup plan (video recording of demo)
2. Create Demo Data
    - Use realistic but simple examples
    - Prepare specific queries that showcase functionality
    - Ensure data loads quickly
3. Rehearse Multiple Times
    - Practice with timer
    - Anticipate technical glitches
    - Prepare backup explanations if demo fails

During Demo

1. Start with Context
    - "I'm now showing the business user experience..."
    - "This is what the IT admin sees..."
2. Narrate Your Actions
    - "I'm typing a question in natural language..."
    - "Notice how the system automatically does x,y,z..."
3. Highlight Key Features
    - Point out governance controls
    - Show conversational interface responding
    - Emphasize user-friendly aspects

4. Handle Issues Gracefully
    - If something breaks, explain what should happen
    - Move to backup plan quickly
    - Don't spend time debugging live

📹 Virtual Presentation Setup (If Applicable)

If presenting virtually via Zoom:

Technical Requirements:

- Stable internet connection (minimum 5 Mbps upload)
- Webcam enabled (for presenter)
- High-quality microphone
- Screen sharing tested in advance

Best Practices:

- Join 15 minutes early to test
- Close unnecessary applications
- Turn off notifications
- Use headphones to avoid echo
- Have good lighting for video
- Professional background (or USU virtual background)

Screen Sharing Strategy:

- Share specific application, not entire screen
- Close sensitive tabs/applications
- Increase font size for readability
- Test screen share before your turn

---

10. Support Resources

🆘 During Hackathon Support

On-Site Support Team

Support Type	Contact Method	Availability
Technical Help Desk	Hackathon administrators	Event hours
USU Organizer Staff	Registration desk	Event hours

📚 Additional Resources

Databricks Community Resources

1. Databricks Community Forums
    - https://community.databricks.com/
    - Search existing questions
    - Post new questions (response within 24 hours typically)
2. Databricks Community Edition Migration Guide
    - https://docs.databricks.com/aws/en/getting-started/ce-migration
    - Helpful for understanding Free Edition features
3. Databricks Documentation
    - https://docs.databricks.com/
    - Comprehensive technical documentation

GitHub Repositories

1. Databricks Apps Examples
    - Search "databricks apps" on GitHub

    - Community-contributed samples
2. Chatbot Templates
    - LangChain examples
    - Gradio chatbot templates
    - Streamlit conversational UI examples

AI/IDE Resources

1. Kiro IDE Documentation
    - https://kiro.dev/
    - Powers documentation
    - Blog and tutorials
2. Cursor AI (Alternative)
    - https://cursor.sh/
    - AI-powered code editor
3. Claude/ChatGPT (For development assistance)
    - Use for code generation help
    - Prompt engineering practice

🎒 What to Bring

Essential Items:

- Laptop (fully charged)
- Laptop charger
- Power strip (outlets limited)
- Mouse (optional but recommended)
- Headphones (for focused coding)
- Water bottle
- Notebook and pen
- Student ID (for check-in)

📧 Contact Information

Databricks Support:

- Free Edition Help:
- https://docs.databricks.com/aws/en/getting-started/free-edition
- Community: community.databricks.com

Appendix A: Quick Start Checklist

Week Before Hackathon

- Register for Databricks Free Edition
- Complete Databricks Fundamentals (2 hours)
- Complete Generative AI Fundamentals (1 hour)
- Watch key video tutorials (1-2 hours)
- Form team and assign roles
- Install necessary tools (Git, Python, IDE)

Day Before Hackathon

- Charge all devices
- Review documentation
- Test Databricks account access
- Prepare laptop development environment
- Get good sleep!

Appendix B: Glossary of Terms

AI/BI Genie - Databricks feature for natural language data queries

Databricks Apps - Platform for building and deploying full-stack data applications within Databricks

Data App Factory - Conceptual framework for enabling business users to create data applications

Delta Lake - Open-source storage layer providing ACID transactions

Governed Data - Data with defined access controls, quality rules, and compliance policies

Gradio - Python library for building ML web applications

Guardrails - Security and policy constraints that govern app creation and deployment

Lakehouse - Data architecture combining data lake and data warehouse capabilities

RAG (Retrieval Augmented Generation) - AI technique for grounding LLM responses in specific data

Steering Document - Template and guidelines that define app creation patterns

Streamlit - Python library for creating data web applications

Unity Catalog - Unified governance solution for data and AI assets in Databricks

---

Appendix C: Sample Use Case Ideas

To inspire your team, here are example use cases:

1. Sales Dashboard Factory

Business User Need: Sales managers want to create custom dashboards for their territories
Solution: Template-based dashboard creator with regional filters, rep performance metrics, and trend analysis. Chat interface for "Show me top performers this quarter."

2. HR Analytics Studio

Business User Need: HR teams need employee dashboards for different departments
Solution: Pre-built templates for turnover analysis, headcount tracking, and diversity metrics. Conversational queries like "What's our retention rate in Engineering?"

3. Customer Insights Hub

Business User Need: Marketing teams want to analyze customer behavior
Solution: Segmentation dashboards, campaign performance, customer journey visualization. Chat: "Which customer segment has the highest LTV?"

4. Supply Chain Monitor

Business User Need: Operations teams need visibility into inventory and shipments
Solution: Real-time inventory dashboards, supplier scorecards, delay alerts. Chat: "Show me delayed shipments from Supplier X."

5. Financial KPI Tracker

Business User Need: Finance teams need P&L views across business units
Solution: Revenue, expense, and margin dashboards with budget vs. actual. Chat: "What's causing variance in Region Northeast?"

Appendix D: Code Snippets & Examples

Databricks Free Edition: An End-to-End Applied Data Science Workflow:
https://github.com/Berkeley-Data/e2e-data-science

Example 1: Simple Streamlit Data App

python

```python
# app.py
import streamlit as st
from databricks import sql
import pandas as pd
import plotly.express as px

# Page config
st.set_page_config(page_title="Sales Dashboard", layout="wide")

# Connect to Databricks
@st.cache_resource
def get_connection():
    return sql.connect(
        server_hostname="your-workspace.cloud.databricks.com",
        http_path="/sql/1.0/warehouses/your-warehouse-id",
        access_token="your-token"
    )

# Query data
@st.cache_data
def load_data():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM main.default.sales_data")
```

```
    df = pd.DataFrame(cursor.fetchall(), columns=[desc for desc
in cursor.description])
    return df

# Main app
st.title("📊 Sales Dashboard")

# Load data
df = load_data()

# KPIs
col1, col2, col3 = st.columns(3)
col1.metric("Total Sales", f"${df['amount'].sum():,.0f}")
col2.metric("Orders", f"{len(df):,}")
col3.metric("Avg Order", f"${df['amount'].mean():,.0f}")

# Visualizations
fig =
px.bar(df.groupby('category')['amount'].sum().reset_index(),
             x='category', y='amount', title='Sales by
Category')
st.plotly_chart(fig, use_container_width=True)

# Chat interface (simplified)
st.subheader("💬 Ask Questions About Your Data")
question = st.text_input("Enter your question:")
if question:
    # This would integrate with Genie API or LLM
    st.write(f"Processing question: {question}")
```

Example 2: Gradio Chatbot Template

python

```
# app.py
```

```python
import gradio as gr
from databricks import sql
import os

def query_databricks(sql_query):
    """Execute SQL query against Databricks"""
    conn = sql.connect(
        server_hostname=os.environ.get("DATABRICKS_SERVER"),
        http_path=os.environ.get("DATABRICKS_HTTP_PATH"),
        access_token=os.environ.get("DATABRICKS_TOKEN")
    )
    cursor = conn.cursor()
    cursor.execute(sql_query)
    results = cursor.fetchall()
    return results

def chatbot(message, history):
    """Process chat message and return response"""
    # This would use Genie API or LLM to translate natural
language to SQL
    # Simplified example:
    if "total sales" in message.lower():
        results = query_databricks("SELECT SUM(amount) FROM
sales")
        return f"Total sales: ${results:,.2f}"
    else:
        return "I can help you query your data. Try asking about
total sales!"

# Create Gradio interface
demo = gr.ChatInterface(
    chatbot,
    title="Data Chat Assistant",
    description="Ask questions about your data in natural
language",
```

```
    examples=["What are total sales?", "Show me top customers",
"Sales by region"]
)

if __name__ == "__main__":
    demo.launch()
```

Example 3: Unity Catalog Access Control

sql

```sql
-- Create a governed table
CREATE TABLE main.sales.customer_data (
    customer_id STRING,
    name STRING,
    email STRING,
    revenue DECIMAL(10,2),
    region STRING
) USING DELTA;

-- Grant access to specific role
GRANT SELECT ON TABLE main.sales.customer_data TO
`data_analyst_role`;

-- Create row-level security
CREATE OR REPLACE FUNCTION main.sales.filter_by_region(region
STRING)
RETURN SELECT * FROM main.sales.customer_data WHERE region =
current_user_region();

-- Apply row-level security
ALTER TABLE main.sales.customer_data SET ROW FILTER
main.sales.filter_by_region ON (region);

-- Mask sensitive column
```

```
ALTER TABLE main.sales.customer_data ALTER COLUMN email SET MASK
mask_email;
```

Conclusion

This hackathon is your opportunity to shape the future of self-service data applications. By combining Databricks' powerful platform with AI-driven development tools and strong governance frameworks, you'll demonstrate how business users can become citizen developers without compromising security or control.

Remember:

- Prepare thoroughly - Complete pre-requisite training
- Collaborate effectively - Leverage your team's diverse skills
- Build iteratively - Start simple, add complexity
- Focus on governance - Security is not optional
- Deliver value - Business users are your customers

Good luck, and may the best Data App Factory win! 🚀

For questions or support, contact:

Brandon Piliero, Account Executive Brandon.Piliero@databricks.com

Elise Hollowed, Manager, University Alliances Elise.Hollowed@databricks.com

Alex Jorna, Solution Architect Alex.Jorna@databricks.com

Anant Asthana, Solution Architect, Anant.Asthana@databricks.com